2024 8th International Conference on Computational System and Information Technology for Sustainable Solutions (CSITSS) | 979-8-3315-0546-2/24/$31.00 ©2024 IEEE | DOI: 10.1109/CSITSS64042.2024.10816962

Parser for TINY Language

Ch. Durga Priya, Ch. Vikram Raju, A. Issac Daniel, G. Rishik Patel, Meena Belwal
Department of Computer science and Engineering
Amrita School of Computing, Bangalore
Amrita Vishwa Vidyapeetham, India
durgapriyachikkala05@gmail.com, vikramchilukuri2266@gmail.com,
anguluriissacdaniel@gmail.com, rishikgandla1269@gmail.com, b_meena@blr.amrita.edu

***Abstract* - This study provides a novel method to generate a compiler, which translates a program using TINY language. TINY language is a simple programming language which is designed to understand the basic concepts of compiler. A compiler is a type of program which is designed to convert the code written in high-level language into machine language that is understandable by the computer. The procedure for building a parser requires various sub-tasks, there are a list of operations that include parsing or splitting the code into tokens, checking's the code's syntax using grammar rules, understanding, interpreting and optimizing the provided code and generating the execution-ready code by the performance of last assembling. In order to face the challenges of compiler construction, which includes rule-based complex grammar, error handling, and fast tokenization, the python programming language is implemented for developing pyLexer(tokenization) and pyParsing (syntactic analysis) libraries. Lexer is used for matching regular expressions to return corresponding tokens(lexemes), while parser is supposed to check whether the given syntax is according to the grammar rules or not through recursion. Web application is created for this using Streamlit, which provides users with many useful features like text rendering capability, error highlighting and code suggestions.**

Keywords— TINY language, compiler, tokenization, syntactic analysis, lexemes, grammar rules, Streamlit.

I. Introduction

TINY language provides simple and effective platform for understanding compiler construction. A compiler is indispensable, role of the compiler is converting high-level program language into machine language which the computer understands and execute. Constructing a compiler is complex due to several challenges. One of the primary challenges is to tokenize the program code into lexemes while handling operators, identifiers, and keywords. Next is parsing which is translating the language according to the grammar rules by handling ambiguity. Furthermore, reacting to the errors and transmitting the error messages correctly.

To tackle these challenges, python programming language is implemented for constructing the two important components: Lexer and parser. Tokenization handles these challenges by implementing a Lexer which uses regular expressions to tokenize the code. Additionally, parsers use techniques like recursive descent parser, LL parsing or LP parsing to handle the challenges. Furthermore, error handling is implemented in every phase, this includes proving error messages and debugging them. The project aim is to build a user-friendly interface where the input is a program that is coded by users and the compiler will compile the code to generate tokens and parse the code. If there is any error in the code, the compiler will return the error message, which helps in debugging and at the same time it serves as an educational content for learning compiler construction, parsing techniques and also language design.

A web application is implemented using Streamlit, user-friendly interface. This is implemented using python libraries. This interface utilizes a TINY assembler language that converts the code in machine language code when the user inputs the code. It also provides educational resources and the interactive feedback for the users to gain knowledge about the processing of compilers, understanding the regular expressions, parsing techniques and error handling.

The subsequent portions of this paper are organised in the following way: Section 1 provides the introduction by giving an overview of the project. Section 2 a review of all the literature papers is provided. In Section 3 methodology is described which is used for better understanding of the paper. Section 4 discusses about the results in each phase. Section 5 concludes the paper by summarizing some of the key results.

II. Literature Review

To carry out this research, several studies were conducted and some of the key research is discussed in this section.

The work by van Tonder and Claire [1] mentioned algorithmic solution as a bright idea for handling of multi-language syntactical transformation. Two strategies that help solve the drawbacks in error reporting in PEGs are offered as a solution. Maidl at el. [2] introduced another approach figuring the failure point of the farthest failure position and an improvement for PEGs. In another work, strategies of automatic grammar induction and partial parsing techniques are introduced by Zhemin and Sung [3]. Ortin [4] discussed about parser generation tooling for both Lex/Yacc, to native language like SaiSiyat which are unknown. Of course, the innovative presented by Hempel and Chugh [5] demonstrated that by using ANTLR, a student achieved high performance compared to traditional Lex/Yacc approaches. The survey conducted by Vaikunta at el. [6] is a research on techniques in lexical analyzer implementation in the process of compiler construction. An approach by Sridhar and Sanagavarapu [7] allows the user to enter commands in command prompt in a natural language way without the use for

Authorized licensed use limited to: Universidade Tecnologica Federal do Parana. Downloaded on September 25,2026 at 16:23:02 UTC from IEEE Xplore. Restrictions apply.

understanding any programming syntax. Gedikli at el. [8] created new programming learning environment that makes it easier for students to use real language, with simple syntax. In this work Aras at el. [9] implemented fuzzy automata concept to eliminate Lexical errors. Eric Brill [10] provided a novel technique for grammatical analysis of unstructured text using transformational grammar. In another approach Bentley [11] explored the idea of "Little Language" and suggested that language-based program provides more efficacy in design. In the research proposed by Doung et al. [12] discussed an approach for training of dependency parsers with low amount data but shared structure that cuts across language. Idris and Ajayi [13] produced a guide for instructional aid with theoretical and practical knowledge of Compiler construction and Automata theory by creating a programming language with Lex/Flex. A new approach is introduced by Biswajit and Abhishek [14]. GLAP procedure was utilized in building and implementing lexical analyzers and came up with improvements in file access tokenization and symbol table application. In another work by Lesk and Schmidt [15] presented Lex as the token generator for lexical processing. Mossenboak [16] presented a novel approach using Alex, a scanner generator which translated the lexical description of programming language into scanner for that language. Another approach by Kulkarni et al. [17] rule-based expert system, by means of natural language processing, ontology and SPARQL is implemented to represent the code pattern using simple natural language and programming semantic. Another approach by Menier and Lorette [18] is lexical analyzer based on self-organizing feature map (SDM) that was made to handle large volume of words at low latency. A new approach by Cox and Clarke [19] introduced ILA (iterative Lexical Analysis) for parsing compared to traditional methods. In another work Pozza et al. [20] suggested a method for judging and comparing the static analysis tools that are intended to expose buffer overflow bugs in C program. Another method by Meena et al. [21] designed a unique tool that interprets and executes mathematical operations as English like sentences. The tool is developed using Lex and Yacc. In another approach Meena et al. [22] presented a novel approach for evaluating mathematical expressions using Abstract Syntax Tress (AST). Another model proposed by Supriya at el. [23] designed and implemented a recognizer and a parser for a set of rudimentary Telugu sentences using the CYK algorithm. In another work by Ayush Barik et al. presented a state-of-art dependency parser for Hindi language using Graph Neural Network (GNN).

III. METHODOLOGY

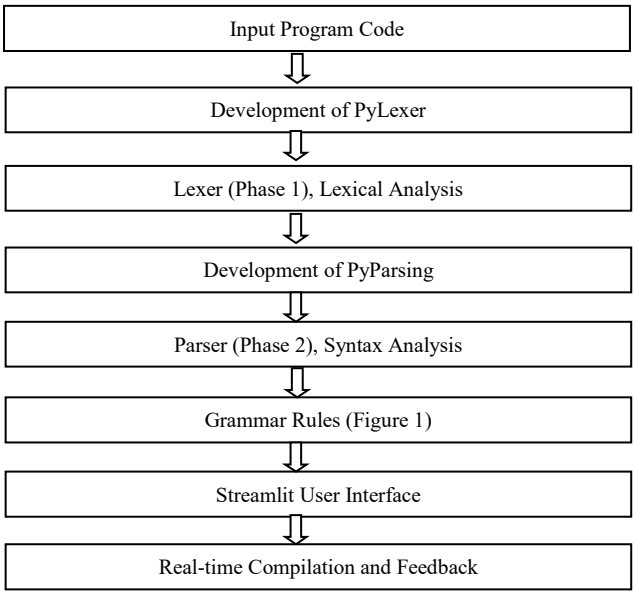


Table.1. Model Architecture.

Flow chart in Table. 1 explains the outline of the model framework. The step-by-step explanation of the model is discussed below:

A. **Input program code:** The first step is writing the program using the TINY program language. The TINY program language has a very simple syntax compared to other programming languages which are complex and contain rich-syntax. The program includes several components such as variable declarations, control statements which are used for loops and conditionals, arithmetic expressions, and Input/output statements. The compiler stores this code in its memory.

B. **Development of PyLexer:** The development of PyLexer starts with initializing various types of tokens that are required by the TINY language to process tokenization. Regurak Expressions are are added to each token to get the type of token within the input. By using regular ecpression libraries in python, it generates the tokens.

C. **Lexer (Phase 1):** The first step is Lexical analysis or tokenization stage where the compiler breaks the program code into sequence of words known as tokens. These special tokens are small units which provides meaning to the code, they are just like keywords, variable names, operators, numbers, and punctuation symbols. The lexer processes the code by character to character, and grouping them based on these tokens by removing whitespaces and comments from the code. Tokenization helpa to transform the input code into a structured format.

The list of all these tokens is sent to next phase by the compiler, which is parsing. Where the tokens

Authorized licensed use limited to: Universidade Tecnologica Federal do Parana. Downloaded on September 25,2026 at 16:23:02 UTC from IEEE Xplore. Restrictions apply.

differentiation and proper organization is done in assistance with grammar rules.

D. Development of PyParsing: The development of Pyparsing starts with defining the grammar rules for the TINY language which is the syntax of the language. It structures the expressions, statements, and control structures. Based on these grammar rules, Recursive Descent parsing or LL parsing is implemented using python programming language. The parser work is to give solutions for understanding syntactical errors and give error free code to generate further.

E. Parser (Phase 2): The compiler receives the list of tokens which were assembles in phase1 (Lexer), and the syntax definitions of these rules are checked with the grammar rules of TINY language. The compiler traces all the tokens from the beginning to the end with the help of grammar rules. If the tokens do not align, the compiler reports with a syntax error and specifically mentions the line number where there is an error in the program.

Grammar Rules: Grammar rules for TINY Language are shown in Figure 1, these rules form the syntax of the programming language. The detailed explanation of these rammar rules are discussed below:

```
Program        ::= StmtList
StmtList       ::= Stmt StmtList | ε
Stmt           ::= AssignStmt | ReadStmt | WriteStmt | IfStmt | WhileStmt | Repeat
AssignStmt     ::= Identifier ':=' Expr ';'
ReadStmt       ::= 'read' Identifier ';'
WriteStmt      ::= 'write' Expr ';'
IfStmt         ::= 'if' Condition 'then' StmtList ElseStmt 'endif'
ElseStmt       ::= 'elseif' Condition 'then' StmtList ElseStmt | 'else' StmtList |
WhileStmt      ::= 'while' Condition 'do' StmtList 'endwhile'
RepeatStmt     ::= 'repeat' StmtList 'until' Condition
Condition      ::= Expr ComparisonOp Expr
ComparisonOp   ::= '<' | '>' | '<=' | '>=' | '==' | '<>'
Expr           ::= Term AddOp Expr | Term
AddOp          ::= '+' | '-'
Term           ::= Factor MulOp Term | Factor
MulOp          ::= '*' | '/'
Factor         ::= '(' Expr ')' | Number | Identifier
Identifier     ::= [A-Za-z]\w*
Number         ::= \d+(\.\d+)?
```

Fig.1. Grammar Rules.

1. *Program ::= StmtList*
   The starting point of grammar, It consists of a list of statements.
2. *StmtList ::= Stmt StmtList | ε*
   Represents list of statements, either followed by another statement or empty.
3. *Stmt ::= AssignStmt | ReadStmt | WriteStmt | IfStmt | WhileStmt | RepeatStmt*
   Defines a statement, which can be an assignment, read, write, if, while or repeat statements.
4. *AssignStmt ::= Identifier ':=' Expr ';'*
   Defines an assignment operator where it assign the value as an expression, ending with a semicolon.
5. *ReadStmt ::= 'read' Identifier ';'*
   Defines an read operation, which reads input into an identifier and ends with semicolon.
6. *WriteStmt ::= 'write' Expr ';'*
   Defines write operation that outputs the value of expression and ends with semicolon.
7. *IfStmt ::= 'if' Condition 'then' StmtList ElseStmt 'endif'*
   Defines an if statement with if or else conditions and ends with endif.
8. *ElseStmt ::= 'elseif' Condition 'then' StmtList ElseStmt | 'else' StmtList | ε*
   Defining an else statement to an if statemrnt, with more else conditions oe empty.
9. *WhileStmt ::= 'while' Condition 'do' StmtList 'endwhile'*
   Defines a while loop with a condition and ends with endwhile.
10. *RepeatStmt ::= 'repeat' StmtList 'until' Condition*
    Defines s repeat-until loop, which iterates the loop until the condition is satisfied.
11. *Condition ::= Expr ComparisonOp Expr*
    Defines boolean operation with two expressions compared to comparision operators.
12. *ComparisonOp ::= '<' | '>' | '<=' | '>=' | '==' | '<>'*
    Defines comparision operators for all the above conditions.
13. *Expr ::= Term AddOp Expr | Term*
    Defines an expression, with addition and subtraction followed by another operation.
14. *AddOp ::= '+' | '-'*
    List of addition and subtraction operations.
15. *Term ::= Factor MulOp Term | Factor*
    Defines term, which can be multiplication or division operator followed by a factor.
16. *MulOp ::= '*' | '/'*
    List of multiplication and division operators.
17. *Factor ::= '(' Expr ')' | Number | Identifier*
    Defining a factor which is enclosed in paranthesis, number or identifier.
18. *Identifier ::= [A-Za-z]\w**
    Defines an identifier, which starts with a letter followed by any alphanumeric characters.
19. *Number ::= \d+(\.\d+)?*
    Defines numeric values, either integer or floating-point number.

These grammar rules outline the structure of TINY program language, explaining how expressions, statements, and conditions are used to form a program. The grammar consists of 19 variables with program as the starting variable.

F. Streamlit User Interface: The compiler keeps the whole process clear-cut through Streamlit. It is a web application where, the user can select if they know about TINY language or not. If the user doesn't know, then it helps in understanding about the TINY language by giving grammar rules and sample program codes. If the user knows about TINY language, then they can directly input the program code and see the program compiling in real-time.

Authorized licensed use limited to: Universidade Tecnologica Federal do Parana. Downloaded on September 25,2026 at 16:23:02 UTC from IEEE Xplore. Restrictions apply.

Tokenization phase's representativeness is depicted, along with the parser output that might include the syntactic errors.

In order to provide a clear and useful approach to compiler construction using the TINY language, the paper presents a well-organized methodology. Through the use of Python to implement Lexer and Parser functions, the study offers educators and students alike an approachable and clear framework. By providing a user-friendly and interactive platform for code input, tokenization visualization, and real-time syntactic error feedback, the Streamlit user interface further improves the learning process. This approach makes the compiler design process transparent and engaging, while also simplifying its intricacies. Notwithstanding the aforementioned drawbacks, the study is successful in producing a useful teaching resource that clarifies compiler construction and encourages a greater comprehension of the underlying ideas.

IV. Results

The results are discussed in this section.

A. **Initiating the system:** To initialize the web application, open command prompt and enter: Streamlit run compiler.py, where compiler.py is filename.

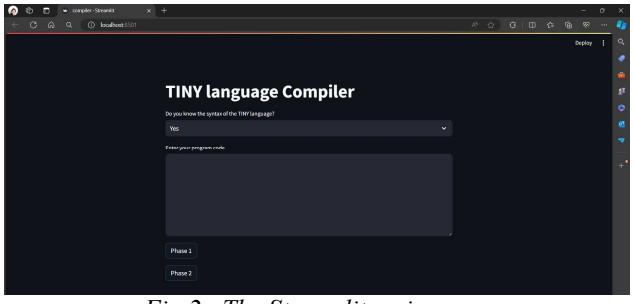


Fig.2. The Streamlit main page.

The Figure 2 depicts how to access Streamlit user interface.

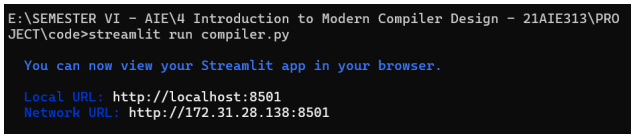


Fig.3. Local and Network URLs.

The details about Local and network URLs for the web application is displayed in command prompt as shown in Figure 3. By using these host addresses, user can directly open the Streamlit application in online directly.

B. **TINY program language:** After opening the web application, the interface asks the user whether the syntax of TINY language is known or not. It gives two options "Yes" or "No" as shown in Figure 3

If it is Yes(known), then the user can input any program code. Then the compiler gives the output for both phase1(Lexer) and phase2(Parser) for the given code.

If it is No(unknown), then it displays the grammar rules which are depicted in Figure 1. I also displays some sample program codes for the users to understand about the TINY compiler language. So, that the users can understand and then generate a code to compile in TINY language.

C. **Basic Programs:** Some basic programs were inputted for users to understand the TINY language.

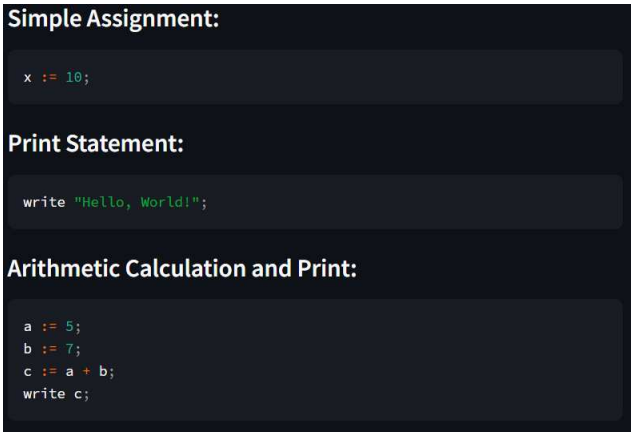


Fig.4. Basic programs.

The Figure 4, shows the programs for simple assignment, print assignment, and arithmetic calculation and printing. There are few more programs which are implemented.

D. **Conditional Statements:** Implementing one of the basic program code that is Conditional statements to compile the code.

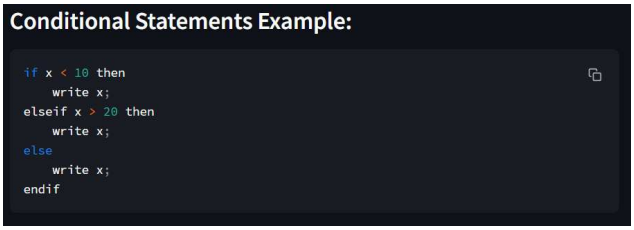


Fig.5. Conditional Statement program code.

Figure 5 displays the program code. For this code, all the stages of tokenization that is token arrays are displayed in results for phase1 (Lexer).



Fig.6. Results for Phase.1(Lexer)

The Figure 6 shows the results for phase 1(Lexer). Further, the results for phase 2(parsing) are also produced along with the message "parsing success!".

Authorized licensed use limited to: Universidade Tecnologica Federal do Parana. Downloaded on September 25,2026 at 16:23:02 UTC from IEEE Xplore. Restrictions apply.

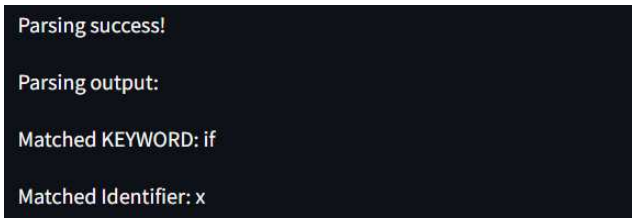


Fig.7. Results for phase 2(Parser)

The Figure 7 displays the results for phase2 (parser).

E. **Print Statement:** Implementing print statement code from the basic programs, it is a single line code which helps in understanding the result better.

```
write "Hello, World!";
```

Fig.8. Print Statement Program Code.

Figure 8 displays the program code. For this code, all the stages of tokenization that is token arrays are displayed in results for phase1 (Lexer).

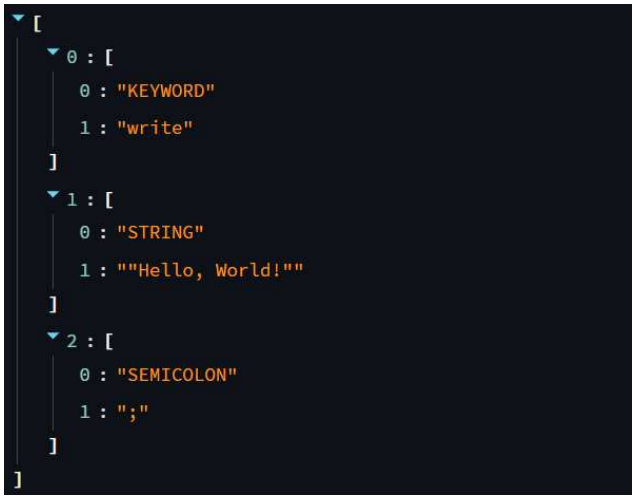


Fig.9. Results for Phase 1(Lexer)

Figure 9 shows the results for phase 1(Lexer). Further, the results for phase 2(parsing) are also produced along with the message "parsing success!". The token array shows that the input code has been successfully tokenized. Each token is separated and it is indicated by token type and its respective value. Which depicts that Lexer has correctly identified the tokens from the input code.

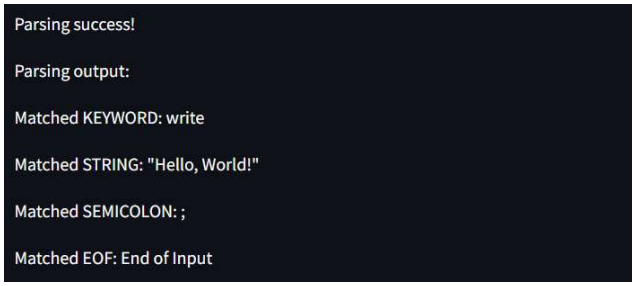


Fig.10. Results for phase 2(Parser)

Figure 10 displays the results for phase2 (parser).

F. **Command Prompt:** The command prompt saves everything that is running in the interface. Figure 12 shows the results for phase2 (parser) that are displayed in command prompt.

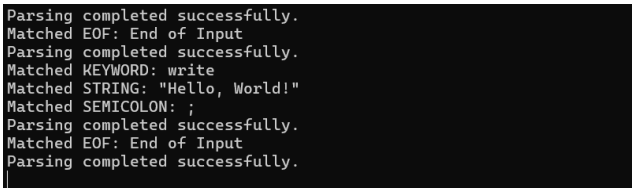


Fig.11. Command prompt results.

Figure 11 Depicts that the parser has successfully matched all those types of tokens from the input code. It identified the 'write/ keyword, a string "Hello, World", and a semicolon ';', followed by end of the input which is end of file (EOF). This shows that the parser has corrected interpreted the syntax of the program. After each phase, End of file (EOF) is also displayed.

G. **Error Handling:** If there is any error in the program code or if the program code is not according to the grammar rules, then the interface gives an error message displaying the error present in which line, such that it helps in correcting the error easily.



Fig.12. Error message.

Figure 12 shows how the interface displays the error message.

By reducing the complexity of compiler design, the paper presents a method of compiler construction using the TINY language, which has substantial educational value. Important features like Python-based Lexer and Parser functions and a Streamlit application for visualization are implemented using this method. Together, these tools handle important compiler building challenges like managing error detection and recovery, handling complex grammar, and guaranteeing effective tokenization. By addressing these issues, the study strengthens compiler development's robustness and educational efficacy.

V. CONCLUSION

In conclusion, this project shows how the compiler for TINY programming language was developed using python programming language, with the integration of lexical analysis, parsing and a user-friendly interface – Streamlit. The compiler tokenizes the small source code and checks the syntax correctness against grammar rules that are already defined in order to generate machine language accurately. The PyLexer and PyParser are used for development of Lexer and parser in python program. The Streamlit interface provided on-time responses and it helps in understanding the TINY program language. And furthermore, the results in both phase1 (Lexer) and phase 2(parser) have compiled successfully. The phase 1(Lexer) has correctly identified the tokens from the input code by

Authorized licensed use limited to: Universidade Tecnologica Federal do Parana. Downloaded on September 25,2026 at 16:23:02 UTC from IEEE Xplore. Restrictions apply.

successfully generating the token array. And also phase 2(Parser) has correctly interpreted the syntax of the program code.

In the future, this project can be expanded by implementing Abstract syntax tree (AST) that can make the source code tokens more simplified and also gives better representations. Additionally, adding another phase of code generation which will translate the AST into machine language will further enhance the compiler functionality. The following enhancements will increase the effectiveness and efficiency of the Tiny language compiler as well as enhance the understanding of compiler construction and optimization methods.

REFERENCES

[1] Van Tonder, Rijnard, and Claire Le Goues. "Lightweight multi-language syntax transformation with parser parser combinators." In Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation, pp. 363-378. 2019.

[2] Maidl, André Murbach, Fabio Mascarenhas, Sérgio Medeiros, and Roberto Ierusalimschy. "Error reporting in parsing expression grammars." Science of Computer Programming 132 (2016): 129-140.

[3] Lin, Zhemin, and Li-May Sung. "Tiny Corpus Applications with Transformation-Based Error-Driven Learning: Evaluations of Automatic Grammar Induction and Partial Parsing of SaiSiyat." In Proceedings of the 18th Pacific Asia Conference on Language, Information and Computation, pp. 197-204. 2004.

[4] Ortin, Francisco, Jose Quiroga, Oscar Rodriguez-Prieto, and Miguel Garcia. "An empirical evaluation of Lex/Yacc and ANTLR parser generation tools." Plos one 17, no. 3 (2022): e0264326.

[5] Hempel, Brian, and Ravi Chugh. "Tiny structure editors for low, low prices!(generating guis from tostring functions)." In 2020 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC), pp. 1-5. IEEE, 2020.

[6] Pai T, Vaikunta, and P. S. Aithal. "A systematic literature review of lexical analyzer implementation techniques in compiler design." International Journal of Applied Engineering and Management Letters (IJAEML) 4, no. 2 (2020): 285-301.

[7] Sridhar, Sashank, and Sowmya Sanagavarapu. "A Compiler-based approach for natural language to code conversion." In 2020 3rd International Conference on Computer and Informatics Engineering (IC2IE), pp. 1-6. IEEE, 2020.

[8] Aras, Sefa, Eyup Gedikli, and Ozcan Ozyurt. "A Framework Based on Compiler Design Techniques for Programming Learning Environments." In 2018 International Conference on Artificial Intelligence and Data Processing (IDAP), pp. 1-6. IEEE, 2018.

[9] Bhosale, Vaishali, and S. R. Chaudhari. "Fuzzy Lexical Analyser: Design and Implementation." International Journal of Computer Applications 123, no. 11 (2015).

[10] Brill, Eric. "Automatic grammar induction and parsing free text: A transformation-based approach." In 31st Annual Meeting of the Association for Computational Linguistics, pp. 259-265. 1993.

[11] Bentley, Jon. "Programming pearls: little languages." Communications of the ACM 29, no. 8 (1986): 711-721.

[12] Duong, Long, Trevor Cohn, Steven Bird, and Paul Cook. "Low resource dependency parsing: Cross-lingual parameter sharing in a neural network parser." In Proceedings of the 53rd annual meeting of the Association for Computational Linguistics and the 7th international joint conference on natural language processing (volume 2: short papers), pp. 845-850. 2015.

[13] Idris, Oyebode, and Adedoyin Olayinka Ajayi. "DOID: A Lexical Analyzer for Understanding Mid-Level Compilation Processes."

[14] Bhowmik, Biswajit, Abhishek Kumar, Abhishek Kumar Jha, and Rajesh Kumar Agrawal. "A New Approach of Complier Design in Context of Lexical Analyzer and Parser Generation for NextGen Languages." International Journal of Computer Applications 6, no. 11 (2010): 21-25.

[15] Lesk, Michael E., and Eric Schmidt. Lex: A lexical analyzer generator. Vol. 39. Murray Hill, NJ: Bell Laboratories, 1975.

[16] Mössenböck, Hanspeter. "Alex—A simple and efficient scanner generator." ACM Sigplan Notices 21, no. 5 (1986): 69-78.

[17] Kulkarni, A. B., S. S. Karandikar, P. A. Bamhore, S. R. Gawade, and D. V. Medhane. "Computational Intelligence Model for Code Generation from Natural Language Problem Statement." In 2018 Fourth International Conference on Computing Communication Control and Automation (ICCUBEA), pp. 1-6. IEEE, 2018.

[18] Menier, Gidas, and Guy Lorette. "Lexical analyzer based on a self-organizing feature map." In Proceedings of the Fourth International Conference on Document Analysis and Recognition, vol. 2, pp. 1067-1071. IEEE, 1997.

[19] Cox, Anthony, and Charles Clarke. "Syntactic approximation using iterative lexical analysis." In 11th IEEE International Workshop on Program Comprehension, 2003., pp. 154-163. IEEE, 2003.

[20] Pozza, Davide, Riccardo Sisto, Luca Durante, and Adriano Valenzano. "Comparing lexical analysis tools for buffer overflow detection in network software." In 2006 1st International Conference on Communication Systems Software & Middleware, pp. 1-7. IEEE, 2006.

[21] Likhith, Arlagadda Naga, Kothuru Gurunadh, Vimal Chinthapalli, and Meena Belwal. "Compiler For Mathematical Operations Using English Like Sentences." In 2023 7th International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS), pp. 1-6. IEEE, 2023.

[22] Pecheti, Shiva Teja, H. M. Basavadeepthi, Nithin Kodurupaka, and Meena Belwal. "Recursive Descent Parser for Abstract Syntax Tree Visualization of Mathematical Expressions." In 2023 7th International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS), pp. 1-6. IEEE, 2023.

[23] Varshini, Surisetty Hima, Gottimukkala Sarayu Varma, and M. Supriya. "A recognizer and parser for basic sentences in telugu using cyk algorithm." In 2023 3rd International Conference on Intelligent Technologies (CONIT), pp. 1-5. IEEE, 2023.

[24] Lolla, Mahimanjali, Ayush Barik, and K. R. Bindu. "GNN Based Trainable Dependency Parser." In 2023 International Conference on Recent Advances in Information Technology for Sustainable Development (ICRAIS), pp. 194-200. IEEE, 2023.

Authorized licensed use limited to: Universidade Tecnologica Federal do Parana. Downloaded on September 25,2026 at 16:23:02 UTC from IEEE Xplore. Restrictions apply.